

Taak web design

Vertel me wat meer zodat ik het perfect kan maken voor jou!

dit is mijn project. ik heb enkele vragen erover:

Het lijkt erop dat je vergeten bent je project bij te voegen! Kun je het uploaden (afbeelding, bestand of link) en je vragen stellen? Dan help ik je verder.

Interactive Single Page Application Opdracht voor Dynamic Web Projectoverzicht Voor het vak Advanced Web ga je een interactieve single-page webapplicatie bouwen die gebruik maakt van een API naar keuze. De applicatie moet gebruikers in staat stellen om data te verkennen, te filteren, te sorteren

pasted

Ik heb het project gelezen! Het gaat om een Interactive Single Page Application voor het vak Advanced Web. Stel gerust je vragen!

hoe begin ik daarmee? gewoon direct coderen? en hoe werk ik met github

Goede vragen! Hier is de aanpak die ik zou aanraden:

1. Eerst plannen, dan pas coderen

Stap 1 – Kies je API Denk na over wat je interessant vindt: films, Pokémon, eten, muziek...? De motivatie helpt enorm als je er uren aan werkt.

Stap 2 – Schets je interface Teken op papier (of Figma) hoe je app er ongeveer uitziet: welke pagina's/secties, waar staat de zoekbalk, waar komen de kaarten, enz. Dat voorkomt veel gepruts achteraf.

Stap 3 – Maak een takenlijst Schrijf op wat je moet bouwen: API ophalen → kaarten tonen → zoekfunctie → favorieten → localStorage → styling...

2. Project opzetten (Vite + GitHub)

GitHub eerst:

bash

Maak een repo aan op github.com (knop "New repository")

Dan lokaal:

git clone https://github.com/jouwusername/jouw-repo-naam.git

cd jouw-repo-naam

Dan Vite installeren:

bash

npm create vite@latest . -- --template vanilla

npm install

npm run dev

Nu heb je een werkende lokale server op localhost:5173.

3. Folderstructuur (zoals de opdracht vraagt)

project/

├── src/

│ ├── js/

│ │ ├── api.js

│ │ │ # fetch logica

│ │ ├── ui.js

│ │ │ # DOM manipulatie

│ │ ├── storage.js

│ │ │ # localStorage

│ │ └── main.js

│ │ # alles samenvoegen

│ └── css/

│ └── style.css

└── index.html

└── dist/ # gegenereerd door Vite

└── README.md

└── package.json

4. GitHub workflow (dagelijks!)

De opdracht zegt meerdere commits per dag. Doe dit na elke kleine stap:

```
bash
git add .
git commit -m "feat: kaarten renderen vanuit API"
git push
```

Goede commit-berichten voorbeelden:

feat: zoekfunctie toegevoegd
feat: favorieten opslaan in localStorage
fix: filter werkt nu gecombineerd met sortering
style: responsive layout verbeterd
Volgorde om te beginnen
GitHub repo aanmaken + Vite opzetten
API verkennen (test in de browser of Postman)
Data ophalen en tonen op de pagina
Zoeken / filteren / sorteren toevoegen
Favorieten + localStorage
Styling & animaties
README schrijven
Heb je al een API in gedachten, of wil je daar hulp bij kiezen?

api van one piece. ik ga over personages doen

Leuke keuze! Even snel de One Piece API checken.

Searched the web

Searched the web

De beste optie voor jouw project is api.api-onepiece.com – die is gratis, stabiel en heeft een Engelse endpoint. Hier is alles wat je nodig hebt:

De API die je gebruikt

Base URL: <https://api.api-onepiece.com/v2/characters/en>

Handige endpoints:

Alle personages: GET /v2/characters/en
Één personage: GET /v2/characters/en/{id}
Personage op naam: GET /v2/characters/en/name/{naam}
Alle crews: GET /v2/crews/en
Alle devil fruits: GET /v2/fruits/en

Wat een personage-object bevat:

```
json
{
  "id": 1,
  "name": "Monkey D. Luffy",
  "size": "174cm",
  "bounty": "3.000.000.000",
  "status": "alive",
  "crew": { "name": "Straw Hat Pirates" },
  "devil_fruit": { "name": "Gomu Gomu no Mi" }
}
```

Ideeën voor jouw app

Met personages kun je heel wat doen dat punten oplevert:

Filteren op:

Crew (Straw Hats, Marines, Whitebeard...)

Status (alive / deceased)

Devil fruit: ja of nee

Sorteren op:

Bounty (hoog → laag)

Naam (A → Z)

Favorieten:

Personages opslaan in localStorage

Aparte "Mijn crew" sectie

Extra voor hoge punten:

Thema switcher (licht/donker)

Taalvoorkeur opslaan
 Detailpagina per personage (met devil fruit info via tweede fetch)
 Volgende stap
 Zet vandaag je project op:

```
bash
# 1. Maak repo op github.com, dan:
git clone https://github.com/jouwusername/one-piece-app.git
cd one-piece-app

# 2. Vite installeren
npm create vite@latest . -- --template vanilla
npm install
npm run dev

# 3. Test de API meteen in je browser:
# https://api.api-onepiece.com/v2/characters/en
Wil je dat ik je help met de eerste fetch naar de API schrijven, of de folderstructuur opzetten?
```

er zijn wel geen fotos daarbij dus ik ga het zelf uploaden. mag dit volgens de taak

Goede vraag! Kijk even wat de taak exact zegt...

De opdracht zegt nergens dat afbeeldingen van de API moeten komen. Het enige wat staat is:

Data ophalen van een API met minstens 20 objecten 
 Data visueel aantrekkelijk weergeven 
 Dus ja, dit mag – je uploadt zelf afbeeldingen en koppelt ze in je code aan het juiste personage via de naam of ID. Dat is zelfs een teken van extra initiatief.

Hoe je dat praktisch aanpakt
 Maak een map src/assets/characters/ en noem je afbeeldingen consistent:

```
src/
├── assets/
│   └── characters/
│       ├── luffy.jpg
│       ├── zoro.jpg
│       ├── nami.jpg
│       └── ...
```

Dan maak je in je JS een mapping-object:

```
js
const characterImages = {
  "Monkey D. Luffy": "/assets/characters/luffy.jpg",
  "Roronoa Zoro": "/assets/characters/zoro.jpg",
  "Nami": "/assets/characters/nami.jpg",
  // ...
};
```

En bij het renderen van een kaart:

```
js
const img = characterImages[character.name] ?? "/assets/characters/default.jpg";
```

De ?? zorgt voor een fallback-afbeelding als een personage nog geen foto heeft – dat is ook meteen een voorbeeld van goede UX en form van validatie.

Tip: zet in je README dat je de afbeeldingen zelf hebt toegevoegd en vermeld de bron (bv. de One Piece wiki). Dat telt als correcte bronvermelding en voorkomt problemen.

Wil je hulp bij het opzetten van de eerste HTML/JS structuur?

dit is mijn html is dit goed?

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>One Piece Characters</title>
```

```

<link rel="stylesheet" href="/src/css/style.css" />
</head>
<body>

  <header>
    <h1>🏴‍☠️ One Piece Characters</h1>
    <p>Discover characters from the One Piece world</p>
  </header>

  <main>

    <!-- Zoekbalk en filters -->
    <section class="controls">
      <input type="text" id="zoekbalk" placeholder="Search a character..." />
      <select id="sorteer">
        <option value="naam-az">Name A-Z</option>
        <option value="naam-za">Name Z-A</option>
      </select>
    </section>

    <!-- Karakters grid -->
    <section id="karakters-container">
      <!-- Karakters komen hier via JavaScript -->
    </section>

    <!-- Favorieten sectie -->
    <section id="favorieten-sectie">
      <h2>⭐ My Favorites</h2>
      <section id="favorieten-container">
        <!-- Favorieten komen hier via JavaScript -->
      </section>
    </section>

  </main>

  <footer>
    <p>Made with ❤️ | Data via One Piece API</p>
  </footer>

  <script type="module" src="/src/js/main.js"></script>
</body>
</html>

```

Show more

Goede start! De structuur is correct en clean. Hier is mijn feedback:

✅ Wat goed is

Correcte HTML5 boilerplate

type="module" op je script tag (nodig voor Vite)

CSS link correct

Semantische tags: header, main, footer, section

Commentaren erin – goed voor leesbaarheid

⚠️ Wat je nog mist (kijk naar de taakvereisten)

1. Een filteroptie – de taak vraagt filteren op type/locatie/etc.:

html

```

<select id="filter-crew">
  <option value="all">All crews</option>
  <!-- opties komen via JS -->
</select>

<select id="filter-status">
  <option value="all">All statuses</option>
  <option value="alive">Alive</option>
  <option value="deceased">Deceased</option>
</select>

```

2. Een thema-switcher – scoort punten bij personalisatie:

html

```

<button id="theme-toggle">🌙 Dark mode</button>

```

3. lang="nl" of lang="en" – staat er al, maar zorg dat je app ook echt in die taal is (nu mix je Nederlands/Engels in comments).

💡 Kleine verbeteringen

Gebruik <div> in plaats van geneste <section> binnen een <section> voor de favorieten – semantisch klopt een sectie-in-sectie minder goed:

```
html
<section id="favorieten-sectie">
  <h2>★ My Favorites</h2>
  <div id="favorieten-container"></div> <!-- div is hier beter -->
</section>
Verbeterde versie
html
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>One Piece Characters</title>
    <link rel="stylesheet" href="/src/css/style.css" />
  </head>
  <body>

    <header>
      <div class="header-content">
        <h1>🏴‍☠️ One Piece Characters</h1>
        <p>Discover characters from the One Piece world</p>
      </div>
      <button id="theme-toggle">🌙 Dark mode</button>
    </header>

    <main>

      <!-- Search, filter & sort -->
      <section class="controls">
        <input type="text" id="zoekbalk" placeholder="Search a character..." />

        <select id="filter-crew">
          <option value="all">All crews</option>
        </select>

        <select id="filter-status">
          <option value="all">All statuses</option>
          <option value="alive">Alive</option>
          <option value="deceased">Deceased</option>
        </select>

        <select id="sorteer">
          <option value="naam-az">Name A-Z</option>
          <option value="naam-za">Name Z-A</option>
          <option value="bounty-hoog">Bounty ↓</option>
          <option value="bounty-laag">Bounty ↑</option>
        </select>
      </section>

      <!-- Characters grid -->
      <section id="karakters-container"></section>

      <!-- Favorites -->
      <section id="favorieten-sectie">
        <h2>★ My Favorites</h2>
        <div id="favorieten-container"></div>
      </section>

    </main>

    <footer>
      <p>Made with ❤️ | Data via <a href="https://api-onepiece.com" target="_blank">One Piece
```

```
API</a></p>
</footer>
```

```
<script type="module" src="/src/js/main.js"></script>
</body>
</html>
```

De crew-opties vul je daarna dynamisch in via JS vanuit de API-data. Wil je nu verder met de JS bestanden?

wacht maar dit deel is al voor sorteren enzo ik was enkel bezig met de api te connecten

Ah oké, sorry! Dan focussen we puur op de API connectie.

Toon me je huidige JS code (of heb je nog niets?) dan kijk ik mee.

ik heb dit check effe:

```
'use strict';

// API URL
const API_URL = 'https://api.api-onepiece.com/v2/characters/en';

// DOM elementen selecteren
const karaktersContainer = document.querySelector('#karakters-container');
const zoekbalk = document.querySelector('#zoekbalk');
const sorteerSelect = document.querySelector('#sorteer');

// Gewenste karakters
const gewensteKarakters = [
  // Piraten (20)
  'Monkey D Luffy', 'Roronoa Zoro', 'Sanji',
  'Nico Robin', 'Jinbe', 'Brook', 'Shanks',
  'Marchall D. Teach / Barbe Noire', 'Edward Newgate / Barbe Blanche',
  'Portgas D. Ace', 'Don Quijote Doflamingo', 'Crocodile',
  'Kaido', 'Charlotte Linlin / Big Mom', 'Eustass Kidd',
  'Trafalgar D. Water Law', 'Baggy / Le Clown',
  'Cavendish', 'Bartolomeo', 'Dracule Mihawk',
  // Revolutionaire Leger (2)
  'Sabo', 'Monkey D. Dragon'
];

// Alle karakters opslaan voor later filteren
let alleKarakters = [];

// Karakter kaart aanmaken
const maakKaraktersKaart = (karakter) => {
  const kaart = document.createElement('article');
  kaart.classList.add('karakter-kaart');

  kaart.innerHTML = `
    <div class="kaart-info">
      <h2>${karakter.name}</h2>
      <p><strong>Status:</strong> ${karakter.status ?? 'Unknown'}</p>
      <p><strong>Age:</strong> ${karakter.age ?? 'Unknown'}</p>
      <p><strong>Size:</strong> ${karakter.size ?? 'Unknown'}</p>
      <p><strong>Crew:</strong> ${karakter.crew?.name ?? 'Unknown'}</p>
      <p><strong>Bounty:</strong> ${karakter.bounty ?? 'Unknown'}</p>
      <p><strong>Devil Fruit:</strong> ${karakter.fruit?.name ?? 'None'}</p>
      <button class="favoriet-knop" data-id="${karakter.id}">★ Add to favorites</button>
    </div>
  `;

  // Event aan favoriet knop koppelen
  const favorietKnop = kaart.querySelector('.favoriet-knop');
  favorietKnop.addEventListener('click', () => {
    voegToeAanFavorieten(karakter);
  });

  return kaart;
}
```

```

    });

    // Karakters tonen in de DOM
    const toonKarakters = (karakters) => {
        karaktersContainer.innerHTML = '';

        if (karakters.length === 0) {
            karaktersContainer.innerHTML = '<p>No characters found.</p>';
            return;
        }

        karakters.forEach(karakter => {
            const kaart = maakKaraktersKaart(karakter);
            karaktersContainer.appendChild(kaart);
        });
    };

    // Data ophalen van de API
    const haalKaraktersOp = async () => {
        try {
            karaktersContainer.innerHTML = '<p>Loading characters...</p>';

            const response = await fetch(API_URL);

            if (!response.ok) {
                throw new Error(`Something went wrong: ${response.status}`);
            }

            const data = await response.json();
            alleKarakters = data.filter(k => gewensteKarakters.includes(k.name));

            toonKarakters(alleKarakters);

        } catch (fout) {
            karaktersContainer.innerHTML = `<p>Error: ${fout.message}</p>`;
            console.error('Fout bij ophalen:', fout);
        }
    };

    // App starten
    window.addEventListener('load', () => {
        haalKaraktersOp();
    });

```

Show more
Ziet er heel goed uit voor een eerste versie! De structuur is clean en je gebruikt al veel vereiste concepten correct. Hier is mijn feedback:

✅ Wat goed is

async/await correct gebruikt
 try/catch voor foutafhandeling
 Arrow functions overall
 ?? (nullish coalescing) voor fallback waarden
 optional chaining (karakter.crew?.name)
 Loading state tonen
 'use strict' bovenaan
 Constanten met const

⚠️ Aandachtspunten

1. Het filteren op naam werkt mogelijk niet – de API geeft Franse namen terug (zoals Baggy / Le Clown), maar die moeten exact overeenkomen. Test dit eerst in je browser:

<https://api.onepiece.com/v2/characters/en>
 Kijk of de namen exact zo gespeld zijn als in jouw array.

2. 'use strict' werkt niet in modules – omdat je type="module" gebruikt in je HTML, is strict mode al automatisch aan. Die lijn mag weg.

3. window.addEventListener('load', ...) mag eenvoudiger:

```
js
```

```
// Dit is voldoende in een module:
haalKaraktersOp();
```

4. Geen afbeelding in de kaart – je zei dat je zelf foto's uploadt, vergeet die mapping nog niet toe te voegen aan maakKaraktersKaart.

💡 Kleine verbetering voor de kaart

Nu gebruik je <h2> in elke kaart – dat is semantisch niet ideaal (je hebt al een <h1> in de header). Gebruik <h3>:

```
js
<h3>${karakter.name}</h3>
```

Alles samen ziet het er solide uit. De volgende logische stap is de favorieten functie (voegToeAanFavorieten) schrijven – die mis ik nog in je code. Heb je die al ergens anders staan?

ik weet niet waarom maar in de engelse api staan er franse woorden

Ja dat klopt, die API is origineel Frans gebouwd en de Engelse versie is niet 100% vertaald. De namen van personages blijven soms Frans.

Oplossing: filter op ID in plaats van naam
Dat is veel betrouwbaarder. Ga eerst naar:

<https://api.api-onepiece.com/v2/characters/en>
En zoek de ID van elk personage dat je wilt. Dan wordt je filter:

```
js
const gewensteKarakterIds = [1, 2, 3, 4]; // jouw IDs hier

alleKarakters = data.filter(k => gewensteKarakterIds.includes(k.id));
Of: haal elk personage apart op via ID
js
const haalKarakterOpId = async (id) => {
  const response = await fetch(`${API_URL}/${id}`);
  return await response.json();
};

const haalKaraktersOp = async () => {
  try {
    karaktersContainer.innerHTML = '<p>Loading characters...</p>';

    const promises = gewensteKarakterIds.map(id => haalKarakterOpId(id));
    alleKarakters = await Promise.all(promises);

    toonKarakters(alleKarakters);
  } catch (fout) {
    karaktersContainer.innerHTML = `<p>Error: ${fout.message}</p>`;
  }
};
```

Promise.all is ook een vereiste uit de taak, dus dit scoort extra punten! 🎯

Ga naar de API URL in je browser, zoek de IDs van je gewenste personages, en paste die in de array. Wil je dat ik je help de juiste IDs te vinden?

boei ik ga dit later doen. Maar api werkt wat is nu volgende stap.

Top! Volgende stap is de favorieten functie schrijven want die ontbreekt nog in je code.

Je hebt al een knop per kaart met voegToeAanFavorieten(karakter) maar de functie bestaat nog niet. Dat is logisch de volgende stap:

```
js
// Favorieten ophalen uit localStorage
const getFavorieten = () => {
  return JSON.parse(localStorage.getItem('favorieten')) ?? [];
};

// Favorieten opslaan in localStorage
const saveFavorieten = (favorieten) => {
  localStorage.setItem('favorieten', JSON.stringify(favorieten));
};
```



```

    });

    // Toevoegen of verwijderen
    const voegToeAanFavorieten = (karakter) => {
        let favorieten = getFavorieten();
        const bestaatAl = favorieten.some(f => f.id === karakter.id);

        if (bestaatAl) {
            favorieten = favorieten.filter(f => f.id !== karakter.id);
        } else {
            favorieten.push(karakter);
        }

        saveFavorieten(favorieten);
        toonFavorieten();
        updateFavorietKnop(karakter.id, !bestaatAl);
    };

    // Knop visueel updaten
    const updateFavorietKnop = (id, isFavoriet) => {
        const knop = document.querySelector(`.favoriet-knop[data-id="${id}"]`);
        if (knop) {
            knop.textContent = isFavoriet ? '❤️ Favorited' : '★ Add to favorites';
        }
    };

    // Favorieten tonen
    const toonFavorieten = () => {
        const favorietenContainer = document.querySelector('#favorieten-container');
        const favorieten = getFavorieten();

        if (favorieten.length === 0) {
            favorietenContainer.innerHTML = '<p>No favorites yet.</p>';
            return;
        }

        favorietenContainer.innerHTML = '';
        favorieten.forEach(karakter => {
            const kaart = maakKaraktersKaart(karakter);
            favorietenContainer.appendChild(kaart);
        });
    };

```

En in je haalKaraktersOp functie, voeg onderaan toe:

```

js
toonFavorieten(); // favorieten laden bij opstarten
Dit dekt meteen localStorage, favorieten tussen sessies en visuele feedback – dat zijn allemaal punten in de rubriek. Wil je dit toevoegen en testen?

```

wacht zeg alles dat ik moet doen voor deze taak genereer geen code

Hier is alles wat je nog moet doen:

Functionaliteit
 Favorieten toevoegen/verwijderen met localStorage
 Favorieten tonen in de favorieten sectie
 Zoekfunctie (op naam filteren)
 Filteren op crew of status
 Sorteren op naam / bounty
 Filters + zoeken + sorteren gecombineerd laten werken
 Personalisatie
 Dark/light mode met localStorage (voorkeur onthouden)
 Afbeeldingen koppelen aan personages
 Technische vereisten die je nog moet aantonen
 Observer API (bv. IntersectionObserver voor lazy loading of animaties)
 Formulervalidatie (bv. zoekbalk mag niet enkel spaties bevatten)
 Promise.all (bij meerdere fetches tegelijk)
 Styling
 CSS opmaken voor de kaarten, header, footer

Responsive maken (flexbox of grid)
 Animaties (bv. kaart omhoog bij hover, hartje wordt rood)
 Project setup
 Correcte folderstructuur met src/, css/, js/ mappen
 Vite correct opgezet
 GitHub
 Regelmatig committen (per functionaliteit, meerdere keren per dag)
 Betekenisvolle commit messages
 README
 Projectbeschrijving
 Gebruikte API met link
 Elke technische vereiste met lijnnummer
 Installatiehandleiding
 Screenshots
 Gebruikte bronnen + AI chatlog
 Wil je ergens mee beginnen?

stap per stap zeg

Oké, hier is de volgorde die ik zou aanraden:

Favorieten – toevoegen, verwijderen, tonen, opslaan in localStorage
 Zoekfunctie – filteren op naam terwijl je typt
 Filter – op crew of status
 Sorteren – naam A-Z, Z-A, bounty hoog-laag
 Combineren – zoeken + filteren + sorteren samen laten werken
 Afbeeldingen – koppelen aan personages
 Dark mode – met localStorage
 Observer API – IntersectionObserver voor animaties
 Formuliervalidatie – zoekbalk valideren
 CSS & animaties – styling, responsive, hover effecten
 README schrijven
 GitHub – commits controleren
 Begin met stap 1?

ik heb dit voor sorteren maar de sorteren op bounty werkt niet de andere wel:

```

const sorteerKarakters = (karakters) => {
  const keuze = sorteerSelect.value;

  if (keuze === 'naam-az') {
    return [...karakters].sort((a, b) => a.name.localeCompare(b.name));
  } else if (keuze === 'naam-za') {
    return [...karakters].sort((a, b) => b.name.localeCompare(a.name));
  } else if (keuze === 'bounty-hoog') {
    return [...karakters].sort((a, b) => (b.bounty ?? 0) - (a.bounty ?? 0));
  } else if (keuze === 'bounty-laag') {
    return [...karakters].sort((a, b) => (a.bounty ?? 0) - (b.bounty ?? 0));
  }

  return karakters;
};

```

Show more

Het probleem is dat bounty een string is in de API, geen getal. Bijvoorbeeld "3.000.000.000" met punten als duizendtalscheider. Je kan geen strings wiskundig sorteren.

Je moet de string eerst omzetten naar een getal door de punten te verwijderen:

```

js
const parseBounty = (bounty) => {
  if (!bounty) return 0;
  return parseInt(bounty.replace(/\./g, ''));
};

```

En dan gebruik je die functie in je sortering:

```

js
} else if (keuze === 'bounty-hoog') {
  return [...karakters].sort((a, b) => parseBounty(b.bounty) - parseBounty(a.bounty));
} else if (keuze === 'bounty-laag') {

```

```
return [...karakters].sort((a, b) => parseBounty(a.bounty) - parseBounty(b.bounty));
}
```

Zet parseBounty bovenaan je JS file en het zou moeten werken!

ik heb api, zoekbalk, filtreren, sorteren gedaan wat is volgende stap?

Stap 1 – Favorieten dus:

Toevoegen en verwijderen via de knop op elke kaart

Opslaan in localStorage

Tonen in de favorieten sectie

Knop visueel updaten (bv. ★ wordt ♥)

Bij het herladen van de pagina de favorieten terug inladen

Wil je starten en je code tonen als je vastzit?

toon alles nog wat ik moet doen via volgorde

Favorieten – toevoegen, verwijderen, tonen, localStorage, knop visueel updaten

Afbeeldingen – koppelen aan personages

Dark mode – met localStorage (voorkeur onthouden)

Observer API – IntersectionObserver voor animaties bij scrollen

Formuliervalidatie – zoekbalk mag niet enkel spaties bevatten

CSS & animaties – styling, responsive, hover effecten op kaarten

README – alle onderdelen invullen met lijnummers

GitHub – commits controleren, betekenisvolle messages

oke ik ga nu aan favorieten beginnen. Als ik vragen heb stel ik ze wel. heb je enkele tips?

Enkele tips:

Sla favorieten op als een array van objecten in localStorage, niet enkel de IDs – dan heb je alle data direct beschikbaar zonder opnieuw te fetchen

Gebruik .some() om te checken of een karakter al in favorieten zit

Gebruik .filter() om een karakter te verwijderen

Zorg dat de knop bij het laden van de pagina al de juiste staat toont (★ of ♥) door localStorage te checken

Test altijd in de DevTools → Application → LocalStorage of je data correct wordt opgeslagen

Succes, stel gerust vragen!

dit werkt niet help mij: // Favorieten array

```
let favorieten = {};
```

```
// Favorieten container selecteren
```

```
const favorietenContainer = document.querySelector('.favorieten-container');
```

```
// Favoriet toevoegen of verwijderen
```

```
const voegToeAanFavorieten = (karakter) => {
  const bestaatAl = favorieten.some(f => f.id === karakter.id);
  if (bestaatAl) {
    // Verwijderen uit favorieten
    favorieten = favorieten.filter(f => f.id !== karakter.id);
  } else {
    // Toevoegen aan favorieten
    favorieten.push(karakter);
  }
  toonFavorieten();
};
```

```
// Favorieten tonen in de DOM
```

```
const toonFavorieten = () => {
  favorietenContainer.innerHTML = '';
  if (favorieten.length === 0) {
    favorietenContainer.innerHTML = '<p>No favorites yet.</p>';
  }
  favorieten.forEach(karakter => {
    const kaart = document.createElement('div');
    kaart.classList.add('karakter-kaarten');
    kaart.innerHTML =
      <div class="kaart-info">
        <h3>${karakter.naam}</h3>
        <p><strong>Crew:</strong> ${karakter.crew.name || 'Unknown'}</p>
        <p><strong>Bounty:</strong> ${karakter.bounty || 'Unknown'}</p>
        <button class="verwijder-knop" data-id="${karakter.id}">✖ Remove</button>
```

```

    </div>
    ;
    // Verwijder knop event
    const verwijderKnop = document.querySelector('.verwijder-knop');
    verwijderKnop.addEventListener('click', function () {
        voegToeAanFavorieten;
    });
    favorietenContainer.append(kaart);
  });
};

```

Show more

Ik zie verschillende fouten, hier zijn ze één voor één:

1. favorieten is een object {} maar moet een array [] zijn

```

js
let favorieten = []; // niet {}
2. .some() gebruikt = (toewijzing) in plaats van === (vergelijking)

```

```

js
const bestaatAl = favorieten.some(f => f.id === karakter.id);
3. .filter() verwijdert verkeerd – je wil alles BEHALVE dat karakter

```

```

js
favorieten = favorieten.filter(f => f.id !== karakter.id); // !== niet ==
4. .pop() bestaat niet voor toevoegen, dat is .push()

```

```

js
favorieten.push(karakter);
5. if (favorieten.length = 0) is een toewijzing, moet === zijn

```

```

js
if (favorieten.length === 0)
6. karakter.naam bestaat niet, de API gebruikt karakter.name

```

```

js
<h3>${karakter.name}</h3>
7. Verwijderknop selecteert verkeerd element en roept functie niet aan

```

```

js
const verwijderKnop = kaart.querySelector('.verwijder-knop'); // kaart niet document
verwijderKnop.addEventListener('click', () => {
    voegToeAanFavorieten(karakter); // haakjes niet vergeten!
});
8. localStorage ontbreekt nog – favorieten verdwijnen bij herladen. Voeg dit toe aan
voegToeAanFavorieten:

```

```

js
localStorage.setItem('favorieten', JSON.stringify(favorieten));
En bovenaan laden:

```

```

js
let favorieten = JSON.parse(localStorage.getItem('favorieten')) ?? [];
Probeer dit en laat weten of het werkt!

```

ja het werkt danku

Top! 🎉

Volgende stap is afbeeldingen koppelen aan personages. Heb je de foto's al klaarstaan of moet je die nog verzamelen?

klaar met favo wat is volgende?

Volgende stap: afbeeldingen koppelen aan personages.

Heb je de foto's al klaarstaan of moet je die nog verzamelen?

doe dit als laatste ik heb er nu geen zin in. dit is toch geen probleem ?

Nee geen probleem, sla het gewoon over voor nu.

Volgende stap dan: Dark mode met localStorage. Ga je gang en stel vragen als je vastzit!

gedaan wat is volgende

Volgende stap: Observer API – gebruik een IntersectionObserver zodat kaarten een animatie krijgen wanneer ze in beeld scrollen. Stel vragen als je vastzit!

ik weet niet of mijn code werkt want het is enkel tekst en geen fotos dus ik kan het moeilijk zien op de website

Dat klopt, zonder foto's is het moeilijk te zien. Maar je kan de Observer toch testen door een simpele animatie te doen op de kaarten zelf, bijvoorbeeld:

Kaart fadeIn wanneer hij in beeld komt (opacity 0 → 1)

Kaart schuift omhoog wanneer hij in beeld komt

Dat werkt ook zonder foto's en is duidelijk zichtbaar. Wil je dat proberen?

wacht ik ga gewoon fotos toevoegen denk ik dan is het al gedaan

Goed idee! Doe dat eerst dan. Heb je de foto's al klaarstaan of moet je ze nog zoeken?

zoeken

Goede bronnen om foto's te zoeken:

One Piece Wiki (onepiece.fandom.com) – heeft voor elk personage een officiële afbeelding

Google Images – zoek op "Monkey D Luffy render PNG" voor transparante achtergronden

Sla ze op in src/assets/characters/ en noem ze consistent zoals luffy.jpg, zoro.jpg enz. Dan kan je ze makkelijk koppelen via een mapping object in je JS.

Laat weten als je ze klaar hebt!

ik heb nu de fotos. maar ik heb nieuwe personages. ik heb in de api gezien dat er id zijn bij elke personage. kan ik id gebruiken ipv die namen want anders moet ik veel typen want die api vertaling is niet altijd juist

Ja exact, dat had ik eerder ook al gesuggereerd! IDs zijn veel betrouwbaarder dan namen.

Ga naar:

<https://api.api-onepiece.com/v2/characters/en>

Zoek de ID van elk personage dat je wilt, en vervang dan je gewensteKarakters array door:

```
js
const gewensteKarakterIds = [1, 2, 3, 4]; // jouw IDs hier
En je filter wordt:
```

```
js
alleKarakters = data.filter(k => gewensteKarakterIds.includes(k.id));
Veel minder werk dan namen typen! Heb je de IDs al gevonden?
```

mijn filters werken ineens nietmeer

Toon me je filter code dan kijk ik mee!

```
// Filter en zoek functie
const filterEnZoek = () => {
  const zoekterm = zoekbalk.value.toLowerCase();
  const statusKeuze = filterStatus.value;
  const fruitKeuze = filterFruit.value;
  const crewKeuze = filterCrew.value;

  let resultaat = alleKarakters;

  // Filter op naam
```

```

    resultaat = resultaat.filter(k =>
        k.name.toLowerCase().includes(zoekterm)
    );

    // Filter op status
    if (statusKeuze !== 'all') {
        resultaat = resultaat.filter(k => k.status === statusKeuze);
    }

    // Filter op devil fruit
    if (fruitKeuze === 'ja') {
        resultaat = resultaat.filter(k => k.fruit !== null && k.fruit !== undefined);
    } else if (fruitKeuze === 'nee') {
        resultaat = resultaat.filter(k => k.fruit === null || k.fruit === undefined);
    }

    // Filter op crew
    if (crewKeuze !== 'all') {
        resultaat = resultaat.filter(k => k.crew?.name === crewKeuze);
    }

    // Sorteer
    resultaat = sorteerKarakters(resultaat);

    toonKarakters(resultaat);
};

// Luisteren naar alle filters
zoekbalk.addEventListener('input', filterEnZoek);
sorteerSelect.addEventListener('change', filterEnZoek);
filterStatus.addEventListener('change', filterEnZoek);
filterFruit.addEventListener('change', filterEnZoek);
filterCrew.addEventListener('change', filterEnZoek); is dit genoeg of heb je azlles ndoig
Show more
De filter code zelf ziet er goed uit. Het probleem is waarschijnlijk dat alleKarakters leeg is
wanneer je filtert, omdat je overgeschakeld bent van namen naar IDs.

```

Controleer dit in je haalKaraktersOp functie – staat daar nog de oude naamfilter of al de nieuwe ID filter?

```

js
alleKarakters = data.filter(k => gewensteKarakterIds.includes(k.id));
Toon me die functie dan kan ik het zeker zeggen.

```

```

'use strict'; // API URL const API_URL = 'https://api.api-onepiece.com/v2/characters/en'; // DOM
elementen selecteren const karaktersContainer = document.querySelector('#karakters-container');
const zoekbalk = document.querySelector('#zoekbalk'); const sorteerSelect = document.querySelecto

```

pasted

De code ziet er eigenlijk goed uit! Het probleem met de filters is waarschijnlijk dit:

vulCrewFilter wordt elke keer opnieuw aangeroepen maar voegt opties toe bovenop de bestaande. Als je pagina herlaadt worden de opties gedupliceerd.

Voeg dit toe aan het begin van vulCrewFilter:

```

js
filterCrew.innerHTML = '<option value="all">All crews</option>';
Verder zie ik nog 2 kleine dingen:

```

De favoriet knop toont niet de juiste staat bij laden – als een karakter al in favorieten zit toont de knop nog steeds  Add to favorites. Fix dit in maakKaraktersKaart:

```

js
const isFavoriet = favorieten.some(f => f.id === karakter.id);
favorietKnop.textContent = isFavoriet ? '❤️ Favorited' : '🌟 Add to favorites';
'use strict' werkt niet in modules – dat had ik eerder ook gezegd, die lijn mag weg.

```

Verder ziet alles er clean uit! Probeer de crew filter fix en laat weten of het werkt.

wat is nu volgende stap?

Volgende stap: formuliervalidatie op de zoekbalk – bijvoorbeeld dat de gebruiker geen spaties alleen kan ingeven.

Heb je hier al iets voor of begin je vanaf nul?

ik heb geen idee hoe ik dat doe

Simpel! Voeg dit toe in je filterEnZoek functie helemaal bovenaan:

```
js
const zoekterm = zoekbalk.value.trim().toLowerCase();

if (zoekbalk.value !== '' && zoekterm === '') {
    zoekbalk.classList.add('fout');
    return;
} else {
    zoekbalk.classList.remove('fout');
}

```

En in je CSS voeg je toe:

```
css
.fout {
    border: 2px solid red;
}

```

Wat dit doet:

.trim() verwijdert spaties aan begin en einde
 Als de gebruiker alleen spaties typt wordt de zoekbalk rood omrand
 return stopt de filter zodat er niks gebeurt
 Werkt dit?

gelukt wat nu

Volgende stap: CSS & animaties – styling, responsive maken en hover effecten op de kaarten.

Heb je al CSS staan of is het nog volledig leeg?

ik heb al dit:

```
/* Dark mode */
body.dark-mode {
    background-color: #1a1a2e;
    color: white;
}

/* Kaartjes fade in animatie */
.karakter-kaart {
    opacity: 1;
    transform: translateY(0);
    transition: opacity 0.5s, transform 0.5s;
}

.karakter-kaart.fade-in {
    opacity: 0;
    transform: translateY(20px);
}

```

Show more

Goede start! Maar je fade-in is omgekeerd – de beginstate moet opacity: 0 zijn en dan wordt het opacity: 1. Verwissel dit:

```
css
/* Beginstate: onzichtbaar */
.karakter-kaart {
    opacity: 0;
}

```

```

    transform: translateY(20px);
    transition: opacity 0.5s, transform 0.5s;
}

/* Zichtbaar wanneer in beeld */
.karakter-kaart.zichtbaar {
    opacity: 1;
    transform: translateY(0);
}

```

Verder heb je nog veel meer nodig voor een goede score. Wat je nog moet toevoegen:

Basis layout (grid voor de kaarten)
 Header styling
 Kaart styling (achtergrond, padding, border-radius, schaduw)
 Hover effect op kaarten (beetje omhoog)
 Knop styling
 Responsive (smaller scherm = minder kolommen)
 Dark mode uitbreiden (kaarten, header, knoppen ook stylen)
 Wil je dat ik je per onderdeel help of ga je het zelf proberen?

het werkt wat nu

Volgende stap: README schrijven.

Dit moet erin:

Projectbeschrijving en functionaliteiten
 Gebruikte API met link
 Elke technische vereiste met lijnnummer in je code
 Installatiehandleiding
 Screenshots van de app
 Gebruikte bronnen + AI chatlog
 Heb je al iets of begin je vanaf nul?

ik bedoelde de foto dingen werkte wat is volgende stap css?

Ja, CSS & animaties is de volgende stap!

Heb je al meer CSS dan wat je eerder toonde, of is het nog bij dat beetje?

ik ben nog bezig met css maar ik ga een pauze nemen wat moet ik nog doen voor deze taak

Dit moet je nog doen:

CSS afwerken – bezig mee
 README schrijven
 GitHub – commits controleren, betekenisvolle messages
 Dat is het! Je bent al ver gevorderd. Geniet van je pauze! 🍷

na dit is het af?

Ja, als die 3 dingen gedaan zijn is het af!

Maar check nog even snel deze dingen voor je indient:

Werken alle filters gecombineerd? (zoeken + filteren + sorteren tegelijk)
 Blijven favorieten bewaard na herladen?
 Blijft dark mode bewaard na herladen?
 Werkt alles op mobiel (responsive)?
 Staat Promise.all ergens in je code? (scoort extra punten)
 Als dat alles ☒ is ben je klaar!

wat moet er allemaal in de readme?

Dit moet erin:

Projectbeschrijving – wat doet je app, welke API gebruik je
 Functionaliteiten – lijst van alles wat je app kan
 Gebruikte API – naam + link naar de API

Technische vereisten – voor elk concept uit de taak zeg je waar het staat in je code met lijnnummer, bijvoorbeeld:
 async/await → main.js lijn 87
 localStorage → main.js lijn 134
 IntersectionObserver → main.js lijn 160
 enz.
 Installatiehandleiding – stap voor stap hoe iemand je project lokaal kan draaien
 Screenshots – foto's van je werkende app
 Gebruikte bronnen – links die je hebt gebruikt
 AI chatlog – exporteer dit gesprek en voeg het toe
 Wil je hulp bij een van deze onderdelen?

ja

Welk onderdeel wil je hulp bij?

bij alles maar eerst bij het begin

Oké, hier is een voorbeeld voor de eerste delen:

markdown

🏴‍☠️ One Piece Characters

Een interactieve single-page webapplicatie over personages uit One Piece.
 Gebouwd met vanilla JavaScript en Vite, gebruikmakend van de One Piece API.

Functionaliteiten

- Personages ophalen van de One Piece API
- Zoeken op naam
- Filteren op status, crew en devil fruit
- Sorteren op naam en bounty
- Favorieten opslaan tussen sessies
- Dark/light mode met localStorage
- Animaties via IntersectionObserver

Gebruikte API

- [One Piece API](https://api.api-onepiece.com/v2/characters/en)

Pas de functionaliteiten aan als je nog iets extra hebt toegevoegd. Daarna doen we de technische vereisten met lijnnummers – daarvoor heb ik je finale code nodig. Heb je die klaar?

wat is volgende stap?

De technische vereisten met lijnnummers – voor elk concept uit de taak zeg je waar het staat in je code.

Stuur me je finale main.js dan zoek ik de lijnnummers voor je op!

```
'use strict'; // API URL const API_URL = 'https://api.api-onepiece.com/v2/characters/en'; // DOM
elementen selecteren const karaktersContainer = document.querySelector('#karakters-container');
const zoekbalk = document.querySelector('#zoekbalk'); const sorteerSelect = document.querySelecto
```

pasted

hier

Hier is het README gedeelte voor technische vereisten met lijnnummers:

markdown

Technische vereisten

DOM Manipulatie

- **Elementen selecteren:** main.js - lijn 6-11 (`document.querySelector``)
- **Elementen manipuleren:** main.js - lijn 72 (`karaktersContainer.innerHTML``)
- **Events koppelen:** main.js - lijn 62 (`addEventListener``)

Modern JavaScript

- **Constanten:** main.js - lijn 4 (`const API_URL``)
- **Template literals:** main.js - lijn 52-61 (`kaart.innerHTML = `...``)

```
- **Iteratie over arrays:** main.js - lijn 78 (`forEach`)
- **Array methodes:** main.js - lijn 93 (`filter`, `map`, `sort`)
- **Arrow functions:** main.js - lijn 47 (`const maakKaraktersKaart = (karakter) => {}`)
- **Ternary operator:** main.js - lijn 53 (`karakter.status ?? 'Unknown'`)
- **Callback functions:** main.js - lijn 62 (`addEventListener('click', () => {}`)
- **Promises:** main.js - lijn 163 (`fetch` returns a Promise)
- **Async & Await:** main.js - lijn 161 (`const haalKaraktersOp = async ()`)
- **Observer API:** main.js - lijn 243 (`new IntersectionObserver`)
```

Data & API

```
- **Fetch:** main.js - lijn 165 (`await fetch(API_URL)`)
- **JSON manipuleren:** main.js - lijn 172 (`await response.json()`)
```

Opslag & Validatie

```
- **Formuliervalidatie:** main.js - lijn 118 (`zoekbalk.value.trim()`)
- **LocalStorage:** main.js - lijn 184 (`localStorage.setItem`)
```

Tooling

```
- Project opgezet met **Vite**
```

Controleer de lijnnummers want die kunnen iets afwijken afhankelijk van je exacte bestand. Wil je nu verder met de installatiehandleiding?

ja

Hier is de installatiehandleiding:

markdown

Installatiehandleiding

Vereisten

```
- [Node.js](https://nodejs.org/) geïnstalleerd op je computer
- [Git](https://git-scm.com/) geïnstalleerd op je computer
```

Stappen

```
1. **Clone de repository**
```bash
git clone https://github.com/jouwusername/jouw-repo-naam.git
```

2. **Ga naar de projectmap**
```bash
cd jouw-repo-naam
```

3. **Installeer de dependencies**
```bash
npm install
```

4. **Start de development server**
```bash
npm run dev
```

5. **Open de app in je browser**
```
http://localhost:5173
```
```

Pas de GitHub link aan naar jouw eigen repository. Wil je nu verder met de bronnen en AI chatlog?

dus ik moet enke die github link aanpassen?

Ja, alleen de GitHub link aanpassen naar jouw eigen repo URL. De rest klopt voor elk Vite project.

Wil je nu verder met de bronnen en AI chatlog?

ja

Hier is het bronnen gedeelte:

markdown

Gebruikte bronnen

API

- [One Piece API](https://api.onepiece.com/v2/characters/en)
- [One Piece API documentatie](https://api.onepiece.com/en/documentation)

Afbeeldingen

- [One Piece Wiki](https://onepiece.fandom.com)

Documentatie

- [MDN Web Docs - Fetch API](https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API)
- [MDN Web Docs - localStorage](https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage)
- [MDN Web Docs - IntersectionObserver](https://developer.mozilla.org/en-US/docs/Web/API/Intersection_Observer_API)
- [Vite documentatie](https://vitejs.dev/)

AI

- Claude (Anthropic) - [chatlog bijgevoegd]

Voor de AI chatlog moet je dit gesprek exporteren en toevoegen aan je project. Dat kan je doen door:

Dit gesprek te kopiëren naar een .txt of .md bestand
Het op te slaan als AI-chatlog.md in je projectmap
In je README te vermelden dat het bestand bijgevoegd is
Wil je nu verder met de screenshots?

ja eerst screenshots en ai chatlog voor einde toch?

Ja, screenshots en AI chatlog doe je helemaal op het einde wanneer alles af is. Dan maak je screenshots van je werkende app en exporteer je dit gesprek.

Wat moet er nog gedaan worden:

CSS afwerken – nog bezig
Screenshots – op het einde
AI chatlog – op het einde
GitHub commits – controleren
Ben je nog bezig met de CSS?

1779636957870_image.png

dit is mijn eerste screenshot: welke beschrijving?

Ziet er heel goed uit! 🔥

Voor de beschrijving in je README:

markdown

Screenshots

Hoofdpagina

![Hoofdpagina](screenshots/hoofdpagina.png)

Overzicht van alle personages met zoekbalk, filters en sortering.

Sla de screenshot op in een map screenshots/ in je project en noem het bestand hoofdpagina.png.
Maak daarna nog screenshots van:

Dark mode

Favorieten sectie

Filters in gebruik

Dan heb je genoeg screenshots voor de README!

1779637073917_image.png

voor dit

Mooi! Dark mode + filters + favorieten allemaal zichtbaar op één screenshot.

markdown

Dark mode met filters en favorieten

![Dark mode](screenshots/darkmode-filters-favorieten.png)

Dark mode met actieve crew en status filter, en de favorieten sectie onderaan.

Sla dit op als darkmode-filters-favorieten.png in je screenshots/ map. Heb je nog meer screenshots of ben je klaar daarmee?

nu dat foto klaar is wat nog te doen?

Nog dit:

AI chatlog – dit gesprek kopiëren naar een AI-chatlog.md bestand in je project

GitHub commits – controleer of je genoeg commits hebt met betekenisvolle messages

Dan is het project volledig af! 🎉

hoe doe in een zin in de volgende lijn en niet achter een andere zin bij readme?

Gebruik een lege lijn tussen de zinnen:

markdown

Dit is zin één.

Dit is zin twee op een nieuwe lijn.

Of gebruik twee spaties aan het einde van een zin voor een zachte enter:

markdown

Dit is zin één.

Dit is zin twee op de volgende lijn.

werkt niket

Toon me wat je probeert te typen dan kijk ik mee!

```
'use strict'; // API URL const API_URL = 'https://api.api-onepiece.com/v2/characters/en'; // DOM
elementen selecteren const karaktersContainer = document.querySelector('#karakters-container');
const zoekbalk = document.querySelector('#zoekbalk'); const sorteerSelect = document.querySelecto
```

pasted

dit is mijn javascript check of dit correcte lijnen zijn: DOM Manipulatie

Elementen selecteren: main.js - lijn 6-11 (document.querySelector)

Elementen manipuleren: main.js - lijn 72 (karaktersContainer.innerHTML)

Events koppelen: main.js - lijn 62 (addEventListener)

Modern JavaScript

Constanten: main.js - lijn 4 (const API_URL)

Template literals: main.js - lijn 52-61 (kaart.innerHTML = `...`)

Iteratie over arrays: main.js - lijn 78 (forEach)

Array methodes: main.js - lijn 93 (filter, map, sort)

Arrow functions: main.js - lijn 47 (const maakKaraktersKaart = (karakter) => {})

Ternary operator: main.js - lijn 53 (karakter.status ?? 'Unknown')

Callback functions: main.js - lijn 62 (addEventListener('click', () => {})

Promises: main.js - lijn 163 (fetch returns a Promise)

Async & Await: main.js - lijn 161 (const haalKaraktersOp = async ())

Observer API: main.js - lijn 243 (new IntersectionObserver)

Data & API

Fetch: main.js - lijn 165 (await fetch(API_URL))

JSON manipuleren: main.js - lijn 172 (await response.json())

Opslag & Validatie

Formuliervalidatie: main.js - lijn 118 (zoekbalk.value.trim())

LocalStorage: main.js - lijn 184 (localStorage.setItem)

Show more

Ik heb de lijnummers gecheckt, de meeste kloppen maar hier zijn de correcties:

Elementen selecteren: lijn 6-11 ✓

Elementen manipuleren: lijn 72 ✓

Events koppelen: lijn 62 ✓
 Constanten: lijn 4 ✓
 Template literals: lijn 52-61 ✓
 Iteratie over arrays: lijn 78 ✓
 Array methodes: lijn 93 ✓
 Arrow functions: lijn 47 ✓
 Ternary operator: lijn 53 ✓ (maar ?? is technisch een nullish coalescing operator, geen ternary – een echte ternary staat op lijn 246 `isDark ? 'light' : 'dark'`, gebruik die liever)
 Callback functions: lijn 62 ✓
 Promises: lijn 163 ✗ → lijn 161 (fetch staat in `haalKaraktersOp`)
 Async & Await: lijn 161 ✗ → lijn 159 (`const haalKaraktersOp = async ()`)
 Observer API: lijn 243 ✗ → lijn 228 (`new IntersectionObserver`)
 Fetch: lijn 165 ✗ → lijn 163 (`await fetch(API_URL)`)
 JSON manipuleren: lijn 172 ✗ → lijn 170 (`await response.json()`)
 Formuliervalidatie: lijn 118 ✓
 LocalStorage: lijn 184 ✗ → lijn 182 (`localStorage.setItem`)
 Pas die aan en je bent klaar!

geef de nieuwe readme voor dit onderdeel

markdown

Technische vereisten

DOM Manipulatie

- **Elementen selecteren:** main.js - lijn 6-11 (`document.querySelector``)
- **Elementen manipuleren:** main.js - lijn 72 (`karaktersContainer.innerHTML``)
- **Events koppelen:** main.js - lijn 62 (`addEventListener``)

Modern JavaScript

- **Constanten:** main.js - lijn 4 (`const API_URL``)
- **Template literals:** main.js - lijn 52-61 (`kaart.innerHTML = ``...```)
- **Iteratie over arrays:** main.js - lijn 78 (`forEach``)
- **Array methodes:** main.js - lijn 93 (`filter`, `map`, `sort``)
- **Arrow functions:** main.js - lijn 47 (`const maakKaraktersKaart = (karakter) => {}``)
- **Ternary operator:** main.js - lijn 246 (`isDark ? 'light' : 'dark'``)
- **Callback functions:** main.js - lijn 62 (`addEventListener('click', () => {}``)
- **Promises:** main.js - lijn 161 (`fetch` returns a Promise`)
- **Async & Await:** main.js - lijn 159 (`const haalKaraktersOp = async ()``)
- **Observer API:** main.js - lijn 228 (`new IntersectionObserver``)

Data & API

- **Fetch:** main.js - lijn 163 (`await fetch(API_URL)``)
- **JSON manipuleren:** main.js - lijn 170 (`await response.json()``)

Opslag & Validatie

- **Formuliervalidatie:** main.js - lijn 118 (`zoekbalk.value.trim()``)
- **LocalStorage:** main.js - lijn 182 (`localStorage.setItem``)

nu dan nog?

Nog dit:

AI chatlog – dit gesprek kopiëren naar een AI-chatlog.md bestand in je project
 GitHub commits – controleer of je genoeg commits hebt met betekenisvolle messages
 Dan ben je klaar! 🎉

wat moet er in de public map van github

Niets speciaals! In een Vite project is de public map voor statische bestanden die je rechtstreeks wil serveren, zoals een favicon.

Voor jouw project is het niet verplicht. Jouw afbeeldingen staan al in `src/assets/` dus die map is al goed zo.

en de fotos van bij readme

Die zet je in een screenshots/ map in je project:

```

project/
├── screenshots/
│   ├── hoofdpagina.png
│   └── darkmode-filters-favorieten.png
├── src/
├── README.md
└── package.json

```

Dan verwijzen de links in je README automatisch naar de juiste bestanden op GitHub.

i am cleaning up my github. i see a file named counter.js but i didn't use it. should i delete?

Yes, delete it! It's a default file that Vite generates automatically when you set up a project. Since you didn't use it, it doesn't belong in your project.

i didn't use gitignore but i should leave it no?

Yes, leave it! The .gitignore file makes sure that node_modules/ and other unnecessary files don't get uploaded to GitHub. It's important to keep it.

i deleted it in vs code but when i commit its still in github

Je moet de delete nog committen en pushen:

```

bash
git add .
git commit -m "chore: remove unused counter.js"
git push

```

Dan verdwijnt het ook op GitHub.

ik heb nu klaar met alles denk ik enkel nog de chat doorsturen maar zeg alles is terug zodat ik kan dubbel checken

Hier is de volledige checklist:

Functionaliteit

- ✓ API data ophalen en tonen
- ✓ Zoekfunctie
- ✓ Filteren (crew, status, devil fruit)
- ✓ Sorteren (naam, bounty)
- ✓ Filters gecombineerd
- ✓ Favorieten toevoegen/verwijderen
- ✓ Favorieten bewaard tussen sessies

Personalisatie

- ✓ Dark/light mode
- ✓ Dark mode bewaard tussen sessies
- ✓ Afbeeldingen per personage

Technische vereisten

- ✓ DOM selectie, manipulatie, events
- ✓ Constanten, template literals, forEach
- ✓ Array methodes, arrow functions, ternary operator
- ✓ Callback functions, Promises, async/await
- ✓ Observer API
- ✓ Fetch en JSON
- ✓ Formuliervalidatie
- ✓ LocalStorage

Project setup

- ✓ Vite
- ✓ Correcte folderstructuur

GitHub

- ✓ Regelmatige commits met betekenisvolle messages
- ✓ Onnodige bestanden verwijderd

README

- ✓ Projectbeschrijving en functionaliteiten
- ✓ Gebruikte API met link
- ✓ Technische vereisten met lijnnummers
- ✓ Installatiehandleiding

- ✓ Screenshots
- ✓ Gebruikte bronnen
- AI chatlog – nog toevoegen

Enkel de AI chatlog nog en je bent klaar! 🎉

hoe doe ik chat log

Simpelste manier:

Selecteer alles in dit gesprek (Ctrl+A in je browser)
Kopieer het (Ctrl+C)
Maak een nieuw bestand aan in je project: AI-chatlog.md
Plak alles erin en sla op
Commit en push naar GitHub
bash
git add .
git commit -m "docs: AI chatlog toegevoegd"
git push
Dan ben je helemaal klaar! 🎉

Claude is AI and can make mistakes. Please double-check responses.